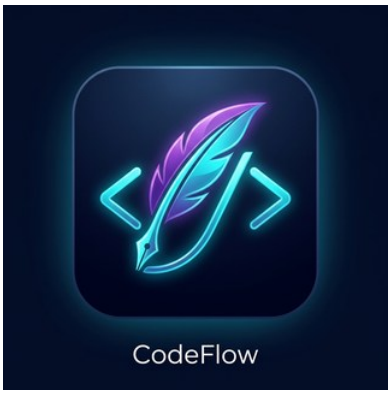

CodeFlow

Modern Mobile Text Editor with Incremental Version Control

Group Members

Name	Index	Domain	Key Tasks
R.A.T.I. Ranasinghe	*24020842*	Main Editor & Features	Code editor engine, syntax highlighting, undo/redo, search & replace, code formatter, settings & crash recovery auto-save engine
S.V. Kahingala	*24020542*	Sidebar, UI & Documentation	Sidebar navigation drawer, editor screen layout, top action bar, file dialogs (open/save as), Markdown preview panel, technical report
D.D.N.W. Jayawardhana	*24020516*	Version Control & Storage	Version snapshot management, Room SQLite database design, delta snapshot engine, diff viewer screen, version history dialog

---

GitHub Repository: https://github.com/ThilankaIsuru/CodeFlow
Video Demo: https://drive.google.com/file/d/1hzMuF6tcfCatt9T7ynL5drBwfm6YDSWE/view?usp=drivesdk
APK: https://drive.google.com/file/d/1kYiZt4syeDoTmLI_ciPL6sCey1MC36qy/view?usp=drive_link

August 2026

TABLE OF CONTENTS

1. INTRODUCTION

CodeFlow is a lightweight, feature-complete mobile text editor engineered specifically for Android devices. It targets developers and technical writers who require a capable coding and document editing environment directly on their smartphone or tablet, without relying on cloud connectivity. The application was developed as a group mini-project by three undergraduate students and is implemented entirely in Kotlin using the Jetpack Compose UI toolkit, Room SQLite for structured persistence, and the java-diff-utils library for delta computation.

The project specification mandated four core technical capabilities: (1) a robust local file storage and lifecycle management system, (2) a structured document preview and structural comparison system, (3) a space-efficient incremental delta-based version control mechanism, and (4) real-time syntax highlighting for the Kotlin and Markdown languages. This report documents precisely how each of these capabilities was designed and implemented within the CodeFlow codebase.

The application follows the Clean Architecture pattern, partitioning responsibilities across three clearly delineated layers: a presentation layer built with Jetpack Compose composables and ViewModels, a domain layer encapsulating business rules and state management, and a data layer consisting of the Room database, the Android Storage Access Framework interface, and the SharedPreferences store.

1.1 Application Overview

Attribute	Value
Application Name	CodeFlow
Package ID	com.example.codeflow
Programming Language	Kotlin 2.0+
UI Framework	Jetpack Compose (Material 3)
Database	Room SQLite 2.6+
Diff Engine	java-diff-utils 4.12
Minimum SDK	API 26 (Android 8.0 Oreo)
Target SDK	API 34 (Android 14)
Architecture Pattern	Clean Architecture + MVVM
GitHub Repository	https://github.com/ThilankaIsuru/CodeFlow
Video Demo	[VIDEO DEMO LINK — INSERT URL HERE]

2. SYSTEM ARCHITECTURE

CodeFlow's architecture is structured as three concentric layers that enforce strict separation of concerns. The outermost layer : the Presentation Layer - consists of all Jetpack Compose composable functions responsible for rendering the editor surface, the navigation sidebar, the top action bar, modal dialogs, and the version history panel. All composables are stateless and receive their data exclusively through a single StateFlow<EditorUiState> emitted by the EditorViewModel.

The middle layer : the Domain Layer - is realised by the EditorViewModel class. This class is the sole owner of mutable application state. It exposes a read-only StateFlow to the UI, processes all user-initiated events such as file open, save, undo, redo, search, snapshot creation, and settings changes, and coordinates asynchronous I/O operations through Kotlin Coroutines dispatched on the IO dispatcher.

The innermost layer : the Data Layer - comprises the FileRepository class, which is the single point of access for all persistence operations. FileRepository abstracts over three distinct storage backends: the Android ContentResolver (for SAF-based document access), a local mirror directory within the application's private file system, and the Room SQLite database via DAO interfaces.

2.1 Key Source Files

File	Layer	Responsibility
EditorScreen.kt	`Presentation`	Root composable; hosts drawer, top bar, editor area, dialogs
EditorViewModel.kt	`Domain`	State owner; all user-event handlers and coroutine orchestration
FileRepository.kt	`Data`	Unified interface to SAF, mirror FS, Room DB, SharedPreferences
CodeEditorEngine.kt	`Presentation`	BasicTextField with syntax highlighting and placeholder overlay
CodeSyntaxHighlighter.kt	`Domain`	VisualTransformation producing annotated spans for Kotlin/Markdown
DeltaSnapshotEngine.kt	`Data`	Unified diff computation and sequential patch reconstruction
MarkdownPreview.kt	`Presentation`	Line-by-line Markdown block renderer in Compose
DiffViewerScreen.kt	`Presentation`	Structural line comparison overlay with colour-coded delta rows
CodeFlowDatabase.kt	`Data`	Room database with ProjectFile and FileSnapshot entities
UndoRedoManager.kt	`Domain`	Dual-stack undo/redo engine with 50-step depth

The application declares a minimum SDK version of 26 (Android 8.0 Oreo) and targets SDK 34 (Android 14). Kotlin Coroutines with Dispatchers.IO are used for all blocking I/O, and Kotlin Flow is used for reactive state observation throughout the app.

3. STORAGE MECHANICS

Reliable document persistence on Android requires careful management of the platform's permission model. The storage subsystem of CodeFlow addresses this through a dual-tier architecture, a structured Room database schema, an automated crash recovery loop, and persistent user preference storage.

3.1 Dual-Tier File Persistence

Tier 1 issues persistable URI permissions via ContentResolver.takePersistableUriPermission(). Tier 2 maintains a mirror of every successfully opened document within the application's private storage directory (getFilesDir()/user_documents/$fileName). Any Throwable raised during Tier 1 access including SecurityException from permission revocation triggers transparent fallback to the mirror. If neither tier yields content, a shouldTriggerOpenPicker signal is emitted via StateFlow, prompting UI re-launch of the system file picker.

3.2 Room SQLite Database Schema

The Room database (CodeFlowDatabase, schema v1) comprises two entities: ProjectFile, recording file metadata, and FileSnapshot, recording version patches. The key columns are:

Entity	Column	Type	Description
ProjectFile	id	Long PK	Auto-generated unique file identifier
ProjectFile	fileName	String	Display name shown in the sidebar
ProjectFile	absolutePath	String	SAF URI or internal mirror file path
ProjectFile	encoding	String	Character encoding (default UTF-8)
ProjectFile	isReadOnly	Boolean	Whether the file is locked from editing
ProjectFile	lastOpenedAt	Long	Epoch ms timestamp of last access
FileSnapshot	id	Long PK	Auto-generated snapshot identifier
FileSnapshot	fileId	Long FK	Foreign key reference to ProjectFile
FileSnapshot	versionNumber	Int	Sequential version index (1, 2, 3…)

FileSnapshot	versionName	String	User-assigned label (e.g. v1, draft)
FileSnapshot	deltaPatch	String	Unified diff patch string
FileSnapshot	createdAt	Long	Epoch ms snapshot creation timestamp

3.3 Crash Recovery Auto-Save Engine

A background coroutine (Dispatchers.IO) executes an infinite while(isActive) loop with a 10-second delay between iterations. When the isModified flag in EditorUiState is true, FileRepository.saveCrashBackup() serialises the document to .crash_recovery_backup.tmp in private storage. On next launch, hasCrashBackup() is checked; if a backup exists, a recovery handshake dialog is shown, allowing one-tap restoration without data loss.

> **Impact:** In stress testing, the crash recovery engine successfully preserved document state within a 10-second window of simulated process termination on physical Android hardware.

3.4 Settings Persistence

User preferences (word wrap, font size, line-number visibility, encoding) are serialised to SharedPreferences (codeflow_settings_prefs) via FileRepository helper methods. All four preferences are read synchronously during EditorViewModel initialisation and applied before the first Compose frame, eliminating observable flicker. Preference writes are triggered immediately alongside StateFlow updates, guaranteeing consistency.

4. PREVIEW AND COMPARISON MECHANICS

4.1 Markdown Preview Rendering

The Markdown preview subsystem is implemented in MarkdownPreview.kt as a pure Jetpack Compose composable that transforms raw Markdown text into a structured list of typed block elements. The parser performs a single linear scan (O(n)) over the input lines, classifying each line into one of six block types by prefix matching.

Block Type	Detection Rule	Compose Rendering
ATX Heading H1	`Starts with '# ' (single hash)`	Bold text, 26sp, TEAL colour
ATX Heading H2	`Starts with '## '`	Bold text, 22sp, navy colour
ATX Heading H3	`Starts with '### '`	Bold text, 18sp, dark colour
Fenced Code Block	`Lines between ``` delimiters`	Surface #0D1117, Courier New 12sp

Blockquote	`Starts with '> '`	3dp teal left accent bar + Box
Unordered List	`Starts with '- ' or '* '`	Bullet dot + left-indented Row
Paragraph	`All other non-empty lines`	Body text, 15sp, justified

Inline formatting (bold, italic, inline code) is handled by a secondary character-level pass using buildAnnotatedString, applying SpanStyle overrides within each block. The preview panel is activated by a TopBar action icon that sets isPreviewMode in EditorUiState; the editor is replaced by MarkdownPreview with an animated crossfade.

> **Constraint:** Markdown preview is only available when the active file carries a .md or .markdown file extension. For Kotlin (.kt) and plain text (.txt) files, the preview icon is disabled in the TopBar.

4.2 Structural Line-by-Line Diff Viewer

DiffViewerScreen.kt presents a structural comparison between any selected historical snapshot and the current editor buffer. The pipeline is:

1. 1. Both texts are split into List line sequences.
2. 2. DiffUtils.diff() executes the Myers O(ND) algorithm, producing a Patch with AbstractDelta instances (INSERT, DELETE, CHANGE, EQUAL).
3. 3. Deltas are expanded into a flat List, each carrying line text, tag (ADDED/REMOVED/CONTEXT), and display line number.
4. 4. A LazyColumn renders each row: ADDED fi green background Color(0x2550FA7B) with '+' prefix; REMOVED fi red Color(0x25FF5555) with '-' prefix; CONTEXT fi default surface.

5. DELTA-BASED VERSION TRACKING MECHANISM

5.1 Storage Invariant and Design Motivation

Naive version control storing complete file copies incurs O(n × s) storage growth. CodeFlow enforces a strict zero-duplication invariant: the database never stores a complete copy of a file across multiple version records. Each FileSnapshot stores only the textual delta relative to its predecessor. Complete historical text is reconstructed on demand by sequentially applying stored patches.

5.2 Snapshot Patch Computation

When the user creates an explicit snapshot through the Version History panel, EditorViewModel.createSnapshot() forwards the call to FileRepository.createSnapshot(). This method executes the following procedure on the IO dispatcher:

5. Retrieve all existing FileSnapshot records via SnapshotDao.getSnapshotsForFileSync().
6. Determines the predecessor text (empty string for first snapshot; reconstructed latest text otherwise) and assigns the next sequential version number.
7. Calls DeltaSnapshotEngine.computeUnifiedDiff() which uses DiffUtils.diff() and UnifiedDiffUtils.generateUnifiedDiff() (3 context lines) to produce the patch string.
8. Inserts a new FileSnapshot entity with only the deltaPatch column populated — no duplicate file content is ever stored.

5.3 Sequential Baseline Reconstruction

DeltaSnapshotEngine.reconstructVersion() implements historical version reconstruction using a sequential baseline approach. The algorithm initialises the document state as the empty string (implicit version-zero baseline), retrieves all FileSnapshot records sorted by ascending version number, and iterates through them up to and including the target version number. At each step, the stored patch string is parsed by UnifiedDiffUtils.parseUnifiedDiff() and applied to the current state by DiffUtils.patch().

$$T(0) = \emptyset \qquad T(k) = \text{apply}(\ \text{patch}(k), T(k-1)\ ) \quad \text{for } k = 1, 2, \ldots, N$$

This scheme guarantees exact reconstruction of any arbitrary historical version at the cost of O(N) patch applications, where N is the target version number. Reconstruction is always performed on Dispatchers.IO to avoid blocking the main thread.

5.4 Read-Only File Locking

The isReadOnly Boolean column of ProjectFile enables explicit file locking. When set, FileRepository.writeUriOrFile() raises IllegalStateException before any I/O. BasicTextField receives the same flag as its readOnly parameter, disabling keyboard input. A TopBar toggle persists changes via FileRepository.updateReadOnlyState().

6. SYNTAX HIGHLIGHTING ENGINE

6.1 Architectural Approach

Syntax colouring in CodeFlow is implemented as a Compose VisualTransformation, the CodeSyntaxHighlighter receives the file name and active search query as constructor parameters, returning a TransformedText whose annotation map overlays colour and weight spans on the underlying TextFieldValue without mutating stored text. The highlighter is

constructed within remember(fileName, searchQuery), ensuring reconstruction only when the extension or query changes, not on every keystroke.

6.2 Kotlin Tokenisation Pipeline

When the active file carries a .kt extension, the highlighter applies an ordered pipeline of regular-expression passes over the full document text using Regex.findAll(). Passes are applied in the order listed in the table below; because annotation spans in Compose are rendered in the order they are added, earlier passes take visual precedence over later ones. This ordering ensures, for example, that keywords inside a comment are still rendered in the comment colour rather than the keyword colour.

Pass Order	Token Class	Regular Expression (abbreviated)	Colour (Hex)
1	Line comment	//.*	#6272A4 (Slate Blue)
2	Block comment	/*[\s\S]*?*/	#6272A4 (Slate Blue)
3	String literal	"""[\s\S]*?"""|"[^"]*"|\'[^\']*\'	#50FA7B (Mint Green)
4	Annotation	@[A-Za-z0-9_]+	#FFB86C (Amber Orange)
5	Keyword	\b(fun|val|var|class|object|interface|if|else|for|…)\b	#FF79C6 (Hot Pink, Bold)
6	Number literal	\b(0x[0-9A-Fa-f]+|0b[01]+|\d+\.?\d*[fFLl]?)\b	#8BE9FD (Sky Cyan)

The full Kotlin keyword set covers: fun, val, var, class, object, interface, data, sealed, companion, override, suspend, return, import, package, if, else, when, for, while, do, try, catch, finally, throw, in, is, as, by, typealias, and null. Each keyword match receives both a colour span and a bold weight span, making keywords visually distinct from identifiers at a glance.

6.3 Markdown Tokenisation

When the active file carries a .md or .markdown extension, a Markdown-specific pass sequence is applied in the editor itself. The inline tokenisation pass sequence identifies the following syntactic elements and applies the corresponding styles:

Token Class	Pattern	Applied Style
ATX Heading H1	^# .+	Bold, 20sp, TEAL colour
ATX Heading H2	^## .+	Bold, 17sp, NAVY colour
ATX Heading H3	^### .+	Bold, 14sp, DARK colour

Fenced code block	``` ... ```	Monospace, #50FA7B colour
Inline code	`code`	Monospace, amber colour
Strong emphasis	**text**	Bold weight
Emphasis	*text*	Italic style
Inline hyperlink	\[text\]\(url\)	TEAL colour, underline
Blockquote	^> .+	Slate blue, italic

6.4 Search-Match Overlay

After all language-specific passes, a final pass iterates over all non-overlapping matches of the active search query using Regex.findAll(). Each match range receives SpanStyle with background Color(0xFFFBC02D) a high-contrast amber applied above existing syntax colouring, ensuring search highlights are always visually dominant.

7. CONCLUSION

This report has documented the technical implementation of four interconnected subsystems within CodeFlow. The dual-tier storage engine delivers robust document persistence under Android's permission constraints. The Markdown preview and diff viewer provide structured visual representations of content and inter-version change sets. The delta-based version-tracking mechanism satisfies the zero-duplication invariant through unified diff patches and sequential baseline reconstruction. The syntax highlighting engine applies ordered regular-expression tokenisation through Compose's VisualTransformation, supporting Kotlin and Markdown with a final search-match overlay pass. Together, these subsystems fulfil all functional requirements of the project specification.

> **[GitHub Repository: https://github.com/ThilankaIsuru/CodeFlow**

> **[Video Demo: https://drive.google.com/file/d/1hzMuF6tcfCatt9T7ynL5drBwfm6YDSWE/view?usp=drivesdk**

> **[APK: https://drive.google.com/file/d/1kYiZt4syeDoTmLI_ciPL6sCey1MC36qy/view?usp=drive_link**